Python 3.12 AsyncIO & High-Performance Coroutines

Presented By samruddhi

Lecture Agenda & Objectives

- • Core Syntax & Architecture
- • Implementation & Code Walkthrough
- • Hands-on Coding Exercise
- • Best Practices & Pitfalls
- • Summary & Next Steps

Python 3.12 AsyncIO & High-Performance Coroutines: Concept 1

- Fundamental logic of Python 3.12 AsyncIO & High-Performance
- • Direct state handling and memory
- • Clean modular implementation

```python
// PYTHON 3.12.8
# Concept: Python 3.12 AsyncIO & High-Performance Corouti...
message = "Python 3.12 AsyncIO & High-Performance Corouti...
print(message)
return message
asyncio.run(demonstrate_concept())
```

Python 3.12 AsyncIO & High-Performance Coroutines: Concept 2

- Fundamental logic of Python 3.12 AsyncIO & High-Performance
- • Direct state handling and memory
- • Clean modular implementation

```
// PYTHON 3.12.8
# Concept: Python 3.12 AsyncIO & High-Performance Corouti...
message = "Python 3.12 AsyncIO & High-Performance Corouti...
print(message)
return message
asyncio.run(demonstrate_concept())
```

Python 3.12 AsyncIO & High-Performance Coroutines: Concept 3

- Fundamental logic of Python 3.12 AsyncIO & High-Performance
- • Direct state handling and memory
- • Clean modular implementation

```
// PYTHON 3.12.8
# Concept: Python 3.12 AsyncIO & High-Performance Corouti...
message = "Python 3.12 AsyncIO & High-Performance Corouti...
print(message)
return message
asyncio.run(demonstrate_concept())
```

Python 3.12 AsyncIO & High-Performance Coroutines: Concept 4

- Fundamental logic of Python 3.12 AsyncIO & High-Performance
- • Direct state handling and memory
- • Clean modular implementation

```
// PYTHON 3.12.8
# Concept: Python 3.12 AsyncIO & High-Performance Corouti...
message = "Python 3.12 AsyncIO & High-Performance Corouti...
print(message)
return message
asyncio.run(demonstrate_concept())
```

Python 3.12 AsyncIO & High-Performance Coroutines: Concept 5

- Fundamental logic of Python 3.12 AsyncIO & High-Performance
- • Direct state handling and memory
- • Clean modular implementation

```python
// PYTHON 3.12.8
# Concept: Python 3.12 AsyncIO & High-Performance Corouti...
message = "Python 3.12 AsyncIO & High-Performance Corouti...
print(message)
return message
asyncio.run(demonstrate_concept())
```

Python 3.12 AsyncIO & High-Performance Coroutines: Concept 6

- Fundamental logic of Python 3.12 AsyncIO & High-Performance
- • Direct state handling and memory
- • Clean modular implementation

```
// PYTHON 3.12.8
# Concept: Python 3.12 AsyncIO & High-Performance Corouti...
message = "Python 3.12 AsyncIO & High-Performance Corouti...
print(message)
return message
asyncio.run(demonstrate_concept())
```

Python 3.12 AsyncIO & High-Performance Coroutines: Concept 7

- Fundamental logic of Python 3.12 AsyncIO & High-Performance
- • Direct state handling and memory
- • Clean modular implementation

```python
// PYTHON 3.12.8
# Concept: Python 3.12 AsyncIO & High-Performance Corouti...
message = "Python 3.12 AsyncIO & High-Performance Corouti...
print(message)
return message
asyncio.run(demonstrate_concept())
```

Hands-On Lab: Python 3.12 AsyncIO & High-Performance Coroutines

- Implement the target logic
- • Pass all edge case assertions
- • Observe expected console output
- • Refactor for optimal performance

```
// PYTHON 3.12.8
# Python 312 AsyncIO  HighPerformance Coroutines Example
def compute_result():
    return 'Result: Verified Output'

print(compute_result())
```

Quick Knowledge Check

- • Fundamental logic of Quick Knowledge Check
- • Direct state handling and memory
- • Clean modular implementation pattern

```
// PYTHON 3.12.8
# Concept: Quick Knowledge Check
message = "Quick Knowledge Check Executed in Python 3.12"
print(message)
return message
asyncio.run(demonstrate_concept())
```

Key Takeaways & Summary

- • Mastered core principles
- • Wrote verified executable code
- • Understood design trade-offs
- • Ready for advanced topics

Thank You & Questions

- • Interactive Q&A Session
- • Hands-on Code Review
- • Repository Links Shared Online
- • Next Lab Preview